Table des matières

Labo 02 OLAP — Le langage SQL sur l'entrepôt 1
Le terrain, en une minute 1
Partie 1 — Lire et trier 2
Partie 2 — Filtrer 4
Partie 3 — Décider et convertir 6
Partie 4 — Le texte 8
Partie 5 — Les dates 10
Partie 6 — Agréger 13
Et ensuite ? 14

Labo 02 OLAP — Le langage SQL sur l'entrepôt

Laboratoire du Cours 02, voie OLAP · DuckDB + AdventureWorks DW · Formation SQL avancé

On reprend les outils de la théorie sur l'entrepôt AdventureWorks DW, dans la ligne de commande DuckDB. Depuis `Projet_SQL/`, ouvrez l'entrepôt par `duckdb adventureworksdw.duckdb`, puis posez le schéma par `USE dbo;`. Ce labo ne fait que lire : rien n'est créé ni modifié. Comptez une heure et demie.

Les deux voies — Ce labo a un jumeau, `Labo_02_OLTP_SQL.md`, qui pose les mêmes questions sur la base transactionnelle dans PostgreSQL. Faites celui-ci d'abord, puis l'autre, et gardez-les ouverts côte à côte : plusieurs requêtes identiques y donnent un résultat différent, et c'est tout l'intérêt.

Tables du jour — `DimProduct`, `DimCustomer`, `DimEmployee`, `DimDate` et `FactInternetSales`. Si le `SHOW TABLES` ci-dessous ne les affiche pas toutes, chargez celles qui manquent comme au Cours 00 : les trois dimensions du milieu n'ont peut-être encore servi à rien.

Le terrain, en une minute

Table	Ce qu'elle contient	Grain	Ce qui servira ici
`DimProduct`	le catalogue, versions comprises	une version de produit, 606 lignes	prix, coût, taille, poids, codes de gamme

© 2026 iAlexMG. Tous droits réservés.

Table	Ce qu'elle contient	Grain	Ce qui servira ici
`DimCustomer`	les clients des ventes en ligne	un client, 18 484 lignes	noms, téléphone, statut familial, distance au travail
`DimEmployee`	le personnel	un employé, 296 lignes	identifiant de connexion, dates de naissance et d'embauche
`DimDate`	le calendrier	un jour, 3 652 lignes de 2005 à 2014	noms et numéros des jours et des mois
`FactInternetSales`	les ventes en ligne	une ligne de commande, 60 398 lignes	dates, numéro de commande, montant

```
USE dbo;
SHOW TABLES;
```

Partie 1 — Lire et trier

1.1 — Calculer, nommer, trier

Théorie : `Theorie_02_SQL.md`, section 1.

Objectif : les cinq produits qui dégagent la plus grosse marge.

```
SELECT EnglishProductName,
       ListPrice,
       StandardCost,
       ListPrice - StandardCost AS marge
FROM dbo.DimProduct
ORDER BY marge DESC
LIMIT 5;
```

Observez. Les cinq lignes portent une vraie marge, la plus forte en tête. Ce résultat tient à un choix du moteur, et c'est là qu'il faut se méfier : 211 produits de l'entrepôt ont un `ListPrice` à `NULL` (Cours 01, section 8), donc une marge à `NULL`, et DuckDB les range en dernier. Sur PostgreSQL, la même requête aurait mis ces 211 lignes vides en tête, et votre top 5 n'aurait contenu aucune marge. Ne laissez pas cette décision au moteur, dites-la :

Piège — L'ordre des `NULL` n'est pas standard, et les deux terrains de la formation ne s'accordent pas. DuckDB les met en dernier dans les deux sens, et son réglage `default_null_order` le dit, à `NULLS_LAST`. PostgreSQL les traite comme la plus grande des valeurs : derniers en ordre croissant, premiers en ordre décroissant. Les deux comportements ont été mesurés. Une requête de classement qui voyage d'un moteur à l'autre sans `NULLS FIRST` ou `NULLS LAST` ne rend donc pas le même palmarès, sans la moindre erreur pour le signaler ; vérifiez ce point sur tout moteur que vous ne connaissez pas encore.

© 2026 iAlexMG. Tous droits réservés.

```sql
SELECT EnglishProductName,
       ListPrice,
       StandardCost,
       ListPrice - StandardCost AS marge
FROM dbo.DimProduct
ORDER BY marge DESC NULLS LAST, EnglishProductName
LIMIT 5;
```

Observez. Le palmarès ne bouge pas, mais il est maintenant écrit et non plus subi : les quatre tailles de Mountain-100 Silver arrivent en tête, avec exactement la même marge de 1 487,84 $ (un prix de 3 399,99 $ pour un coût de 1 912,15 $). Sans la seconde clé de tri, `EnglishProductName`, leur ordre relatif serait laissé au moteur, et la cinquième ligne, une des Mountain-100 Black, pourrait changer d'une exécution à l'autre.

1.2 — Les valeurs distinctes, et un tri qui trompe

Théorie : `Theorie_02_SQL.md`, sections 1 et 3.

Objectif : lister les distances domicile-travail des clients, dans l'ordre.

```sql
SELECT DISTINCT CommuteDistance
FROM dbo.DimCustomer
ORDER BY CommuteDistance;
```

Observez. Cinq valeurs, dans cet ordre : « 0-1 Miles », « 1-2 Miles », « 10+ Miles », « 2-5 Miles », « 5-10 Miles ». La distance la plus longue s'intercale au milieu. La colonne est du texte, et le texte se trie caractère par caractère : « 10+ » commence par « 1 », qui passe devant « 2 ». Le vrai ordre se déclare avec un `CASE` :

```sql
SELECT DISTINCT CommuteDistance,
       CASE CommuteDistance
           WHEN '0-1 Miles'  THEN 1
           WHEN '1-2 Miles'  THEN 2
           WHEN '2-5 Miles'  THEN 3
           WHEN '5-10 Miles' THEN 4
           WHEN '10+ Miles'  THEN 5
       END AS rang
FROM dbo.DimCustomer
ORDER BY rang;
```

1.3 — L'alias dans le `WHERE`

Théorie : `Theorie_02_SQL.md`, section 1.

Objectif : voir ce que DuckDB accepte et que PostgreSQL refuse.

© 2026 iAlexMG. Tous droits réservés.

```sql
SELECT EnglishProductName, ListPrice * 1.05 AS prix_avec_tps
FROM dbo.DimProduct
WHERE prix_avec_tps > 3500;
```

Observez. DuckDB exécute la requête et rend 13 lignes, les Road-150 et les Mountain-100, alors que l'alias `prix_avec_tps` n'existe, en principe, qu'après le `WHERE`. Le labo jumeau vous montrera PostgreSQL refuser la même écriture. Retenez la règle de prudence de la théorie : dans une requête destinée à voyager, répétez l'expression.

1.4 — Garder l'étoile sans tout prendre

Théorie : `Theorie_02_SQL.md`, section 1.

Objectif : lire les clients sans leurs coordonnées personnelles.

```sql
SELECT * EXCLUDE (BirthDate, EmailAddress, Phone, AddressLine1, AddressLine2)
         REPLACE (UPPER(LastName) AS LastName)
FROM dbo.DimCustomer
LIMIT 3;
```

Observez. Les 24 autres colonnes de `DimCustomer` sortent dans leur ordre, et `LastName` arrive en majuscules à sa place, `YANG`, `HUANG`, `TORRES`. La date de naissance, le courriel, le téléphone et l'adresse ne quittent pas la base : une analyse des ventes n'en a pas besoin.

Erreur voulue : nommer une colonne qui n'existe pas.

```sql
SELECT * EXCLUDE (Telephone)
FROM dbo.DimCustomer
LIMIT 1;
```

Observez. `Binder Error: Column "Telephone" in EXCLUDE list not found in FROM clause`. La colonne s'appelle `Phone`. Le moteur refuse au lieu de laisser passer le téléphone, et c'est ce qu'on veut d'un filtre de données personnelles.

Partie 2 — Filtrer

2.1 — La priorité de `AND` sur `OR`

Théorie : `Theorie_02_SQL.md`, section 2.

Objectif : compter les produits rouges ou noirs vendus plus de 1 000 $.

```sql
SELECT COUNT(*) AS sans_parentheses
FROM dbo.DimProduct
WHERE Color = 'Red' OR Color = 'Black' AND ListPrice > 1000;
```


© 2026 iAlexMG. Tous droits réservés.

```sql
SELECT COUNT(*) AS avec_parentheses
FROM dbo.DimProduct
WHERE (Color = 'Red' OR Color = 'Black') AND ListPrice > 1000;
```

Observez. 104 lignes sans parenthèses, 71 avec. La première requête a compté tous les produits rouges, 63 lignes, quel que soit leur prix, puis y a ajouté les noirs chers. Aucune erreur, deux réponses plausibles : c'est exactement le genre de faute qui passe une revue de code.

`IN` règle le cas d'une liste de valeurs sur la même colonne, sans aucune ambiguïté de priorité : `WHERE Color IN ('Red', 'Black', 'Silver')` compte 248 lignes.

2.2 — `BETWEEN`, et le piège qui dort

Théorie : `Theorie_02_SQL.md`, section 2.

Objectif : compter les lignes de vente de janvier 2013.

```sql
SELECT COUNT(*) AS avec_between
FROM dbo.FactInternetSales
WHERE OrderDate BETWEEN '2013-01-01' AND '2013-01-31';

SELECT COUNT(*) AS demi_ouvert
FROM dbo.FactInternetSales
WHERE OrderDate >= '2013-01-01' AND OrderDate < '2013-02-01';
```

Observez. 1 662 lignes des deux côtés, alors que la théorie annonçait un écart. `OrderDate` est pourtant un horodatage. Regardez ses heures :

```sql
SELECT COUNT(*) AS lignes_avec_une_heure
FROM dbo.FactInternetSales
WHERE OrderDate::TIME <> '00:00:00';
```

Zéro. Toutes les commandes sont enregistrées à minuit, et la borne `'2013-01-31'` tombe donc pile sur les 100 lignes du 31 janvier. Le piège est bien là, il dort. Il se réveillera le jour où une source enregistrera l'heure réelle des commandes, et la requête avec `BETWEEN` perdra alors une partie du dernier jour de chaque mois, sans lever d'erreur. L'intervalle demi-ouvert, lui, restera juste.

2.3 — Casse, accents et motifs

Théorie : `Theorie_02_SQL.md`, section 2.

Objectif : trouver les vélos par leur code, et les mois d'août par leur nom.

```sql
SELECT COUNT(*) AS avec_like  FROM dbo.DimProduct WHERE ProductAlternateKey LIKE 'bk-%';
SELECT COUNT(*) AS avec_ilike FROM dbo.DimProduct WHERE ProductAlternateKey ILIKE 'bk-%';
```


© 2026 iAlexMG. Tous droits réservés.

Observez. Aucune ligne avec `LIKE`, 125 avec `ILIKE` : les codes sont écrits en majuscules, `BK-M82S-38`, et `LIKE` distingue la casse. Passons aux accents, sur le calendrier :

```
SELECT COUNT(*) AS sans_accent FROM dbo.DimDate WHERE FrenchMonthName ILIKE 'aout';
SELECT COUNT(*) AS avec_accent FROM dbo.DimDate WHERE FrenchMonthName ILIKE 'août';
```

Observez. Zéro jour sans l'accent, 310 avec, soit les dix mois d'août de 2005 à 2014. `ILIKE` a ignoré la majuscule de « Août », pas son accent.

2.4 — Le filtre qui ne trouve rien

Théorie : `Theorie_02_SQL.md`, sections 2 et 4.

Objectif : compter les produits de la gamme route, codée `R`.

```
SELECT COUNT(*) AS gamme_route
FROM dbo.DimProduct
WHERE ProductLine = 'R';
```

Observez. Zéro. La gamme existe pourtant. Regardez les valeurs, et surtout leur longueur :

```
SELECT ProductLine, LENGTH(ProductLine) AS longueur, COUNT(*) AS produits
FROM dbo.DimProduct
GROUP BY ProductLine
ORDER BY produits DESC;
```

Observez. Chaque code mesure deux caractères : la valeur rangée est `'R '`, suivie d'une espace. Dans la base d'origine, la colonne est déclarée `nchar(2)`, à longueur fixe : le script d'installation de Microsoft le dit (Cours 00), et un type à longueur fixe complète ses valeurs d'espaces. Le fichier exporté les a gardées, le chargement du Cours 00 aussi, et un `VARCHAR` compte chaque espace à la comparaison. Le correctif :

```
SELECT COUNT(*) AS gamme_route
FROM dbo.DimProduct
WHERE TRIM(ProductLine) = 'R';
```

Observez. 162 lignes. Le même piège frappe `Class`, `Style` et les deux codes d'unité : un filtre `WeightUnitMeasureCode = 'LB'` rend zéro ligne, et sa version avec `TRIM` en rend 253. Retenez le réflexe : avant de filtrer sur un code, vérifiez sa longueur.

Partie 3 — Décider et convertir

3.1 — Étiqueter avec `CASE`

Théorie : `Theorie_02_SQL.md`, sections 3 et 7.

Objectif : répartir les clients selon le genre et le statut familial, en clair.

© 2026 iAlexMG. Tous droits réservés.

```sql
SELECT CASE Gender WHEN 'F' THEN 'femme' WHEN 'M' THEN 'homme' END              AS genre,
       CASE MaritalStatus WHEN 'M' THEN 'marié' WHEN 'S' THEN 'célibataire' END AS etat_civil,
       COUNT(*) AS clients
FROM dbo.DimCustomer
GROUP BY ALL
ORDER BY genre, etat_civil;
```

Observez. Quatre groupes : 4 388 femmes célibataires, 4 745 femmes mariées, 4 085 hommes célibataires, 5 266 hommes mariés. `GROUP BY ALL` a regroupé sur les deux colonnes calculées sans qu'on les répète (théorie, section 7). La forme simple du `CASE` convient ici, parce que les deux codes ne sont jamais `NULL`.

Pour un choix à deux branches, il existe un raccourci :

```sql
SELECT IF(HouseOwnerFlag = '1', 'propriétaire', 'locataire') AS logement,
       COUNT(*) AS clients
FROM dbo.DimCustomer
GROUP BY ALL;
```

Observez. 12 502 propriétaires et 5 982 locataires. Notez la comparaison à `'1'`, entre apostrophes : `HouseOwnerFlag` est du texte, pas un booléen ni un nombre.

3.2 — Convertir sans tout faire planter

Théorie : `Theorie_02_SQL.md`, sections 4 et 5.

Objectif : calculer la taille moyenne des produits, dont certaines tailles sont des lettres.

```sql
SELECT AVG(CAST(Size AS INTEGER))
FROM dbo.DimProduct;
```

Observez. Erreur : le moteur s'arrête sur la première taille en lettres, « M » ou « L ». La conversion tolérante :

```sql
SELECT COUNT(*)                                  AS produits,
       COUNT(TRY_CAST(Size AS INTEGER))          AS tailles_numeriques,
       ROUND(AVG(TRY_CAST(Size AS INTEGER)), 1) AS taille_moyenne
FROM dbo.DimProduct;
```

Observez. 606 produits, 254 tailles numériques, une taille moyenne de 49,1. Le calcul est juste, mais comptez ce que `TRY_CAST` a changé en `NULL` sans rien dire :

```sql
SELECT COUNT(*) AS tailles_devenues_null
FROM dbo.DimProduct
WHERE Size IS NOT NULL
  AND TRY_CAST(Size AS INTEGER) IS NULL;
```

Observez. 45 tailles en lettres (M, L, S, XL) sont sorties du calcul. Ici c'est voulu ; ailleurs, ce serait une perte de données invisible.

Même idée sur le numéro de commande, `SO43697`, un préfixe de deux lettres suivi d'un nombre :

© 2026 iAlexMG. Tous droits réservés.

```sql
SELECT MIN(CAST(SUBSTR(SalesOrderNumber, 3) AS INTEGER)) AS premier_numero,
       MAX(CAST(SUBSTR(SalesOrderNumber, 3) AS INTEGER)) AS dernier_numero,
       COUNT(DISTINCT SalesOrderNumber)                  AS commandes
FROM dbo.FactInternetSales;
```

Observez. Des numéros de 43 697 à 75 123, pour 27 659 commandes : 3 768 numéros de la plage n'apparaissent pas dans les ventes en ligne. `SUBSTR(…, 3)` garde tout à partir du troisième caractère, et la conversion ne peut pas échouer, parce que tous les numéros ont la même forme.

3.3 — Diviser et arrondir

Théorie : `Theorie_02_SQL.md`, section 4.

Objectif : la part des clients propriétaires de leur logement.

```sql
SELECT COUNT_IF(HouseOwnerFlag = '1')                              AS proprietaires,
       COUNT(*)                                                    AS clients,
       COUNT_IF(HouseOwnerFlag = '1') / COUNT(*)                   AS part,
       ROUND(100 * COUNT_IF(HouseOwnerFlag = '1') / COUNT(*), 1)   AS pourcentage
FROM dbo.DimCustomer;
```

Observez. Une part de 0,676368… et 67,6 %. Deux entiers divisés l'un par l'autre donnent un décimal sur DuckDB. `COUNT_IF` est un raccourci, que DuckDB a et que PostgreSQL n'a pas ; le Cours 06 enseigne l'écriture portable, l'agrégat conditionnel. La même division rendra 0 dans le labo jumeau.

Le type d'une colonne influe aussi sur une conversion d'unités. Le poids des produits est exprimé tantôt en livres, tantôt en grammes :

```sql
SELECT ROUND(MAX(CASE WHEN WeightUnitMeasureCode = 'G' THEN Weight / 453.592
                      ELSE Weight END), 2)                         AS sans_trim,
       ROUND(MAX(CASE WHEN TRIM(WeightUnitMeasureCode) = 'G' THEN Weight / 453.592
                      ELSE Weight END), 2)                         AS avec_trim
FROM dbo.DimProduct;
```

Observez. Sans `TRIM`, le produit le plus lourd pèserait 1 050 livres : la condition ne reconnaît jamais `'G '`, aucun poids en grammes n'est converti, et une roue de 1 050 grammes passe pour une charge de 1 050 livres. Avec `TRIM`, le plus lourd pèse 30 livres. Le piège de la sous-partie 2.4 vient de fausser un calcul au lieu d'un filtre, et c'est pire : le résultat a l'air d'un nombre.

Partie 4 — Le texte

4.1 — Le nom complet

Théorie : `Theorie_02_SQL.md`, section 5.

Objectif : afficher le nom complet de chaque client.

© 2026 iAlexMG. Tous droits réservés.

```sql
SELECT COUNT(*)                                                AS clients,
       COUNT(FirstName || ' ' || MiddleName || ' ' || LastName) AS noms_complets
FROM dbo.DimCustomer;
```

Observez. 18 484 clients, 10 654 noms complets : 7 830 noms ont disparu, ceux des clients sans second prénom. L'écriture qui résiste :

```sql
SELECT FirstName || COALESCE(' ' || MiddleName, '') || ' ' || LastName AS nom_complet
FROM dbo.DimCustomer
ORDER BY CustomerKey
LIMIT 3;
```

Observez. « Jon V Yang », « Eugene L Huang », « Ruben Torres » : le troisième n'a pas de second prénom et garde son nom.

Portabilité — `CONCAT_WS` fait le même travail en plus court, et saute les arguments `NULL` au lieu de tout annuler : `CONCAT_WS(' ', 'Ruben', NULL, 'Torres')` rend « Ruben Torres » sur DuckDB comme sur PostgreSQL, les deux moteurs de la formation. Ce comportement n'est pas universel, et la documentation d'un moteur inconnu se vérifie sur ce point précis avant de s'y fier : c'est le genre de détail qui ne lève jamais d'erreur, il vide juste une colonne.

4.2 — Deux formats de téléphone

Théorie : `Theorie_02_SQL.md`, sections 3 et 5.

Objectif : ramener tous les téléphones à la forme 999-999-9999.

```sql
SELECT CASE WHEN Phone LIKE '1 (11) %' THEN 'avec indicatif' ELSE 'local' END AS format,
       COUNT(*) AS clients
FROM dbo.DimCustomer
GROUP BY ALL;
```

Observez. 9 461 numéros locaux, comme `717-555-0164`, et 9 023 précédés d'un indicatif, comme `1 (11) 500 555-0162`. Le préfixe `1 (11)` fait toujours sept caractères, suivis de deux blocs séparés par une espace. Pour normaliser, on garde ce qui suit le préfixe et on change l'espace en trait d'union :

```sql
SELECT Phone,
       CASE WHEN Phone LIKE '1 (11) %' THEN REPLACE(SUBSTR(Phone, 8), ' ', '-')
            ELSE Phone
       END AS telephone
FROM dbo.DimCustomer
WHERE CustomerKey = 11000;
```

Observez. `1 (11) 500 555-0162` devient `500-555-0162`. Appliquée à toute la table, la même expression ramène les 18 484 numéros à une seule forme.

4.3 — Découper : le nom de produit et l'identifiant de connexion

Théorie : `Theorie_02_SQL.md`, section 5.

© 2026 iAlexMG. Tous droits réservés.

Objectif : vérifier si ce qui suit la virgule d'un nom de produit est bien sa taille.

```sql
SELECT EnglishProductName,
       TRIM(SPLIT_PART(EnglishProductName, ',', 2)) AS apres_virgule,
       Size
FROM dbo.DimProduct
WHERE EnglishProductName LIKE '%,%'
  AND TRIM(SPLIT_PART(EnglishProductName, ',', 2)) <> COALESCE(Size, '');
```

Observez. Sur 308 noms qui contiennent une virgule, 298 finissent bien par la taille. Les 10 autres finissent par une couleur ou un format : les casques Sport-100 « Red », « Black » et « Blue », et les sacoches « Large ». Une règle de découpage déduite de quelques lignes ne vaut que pour ces lignes ; ici, la vraie taille est dans la colonne `Size`.

L'identifiant de connexion des employés porte un domaine et un nom, séparés par une barre oblique inverse :

```sql
SELECT LoginID,
       SPLIT_PART(LoginID, '\', 2)  AS identifiant,
       SPLIT_PART(LoginID, '\\', 2) AS separateur_double,
       LENGTH('\\')                 AS longueur_du_double
FROM dbo.DimEmployee
LIMIT 3;
```

Observez. `adventure-works\guy1` donne `guy1` avec `'\'`, et une chaîne vide avec `'\\'`. Sur DuckDB, la barre oblique inverse est un caractère ordinaire dans un littéral : `'\'` en contient une, et `'\\'` deux, comme le montre la dernière colonne. Aucun `LoginID` ne contient deux barres de suite, et `SPLIT_PART` rend alors une chaîne vide, sans la moindre erreur.

Portabilité — Le labo jumeau écrit le même `'\'` sur PostgreSQL. Snowflake, d'après sa documentation, se sert au contraire de la barre oblique inverse pour échapper le caractère qui la suit, et y demande `'\\'` pour une barre seule. Une requête qui passe d'un moteur à l'autre sans qu'on double ou dédouble ses barres rend des chaînes vides, sans rien signaler.

Partie 5 — Les dates

5.1 — De la clé à la date, de la date au délai

Théorie : `Theorie_02_SQL.md`, sections 4 et 6.

Objectif : vérifier la clé de date, puis mesurer les délais d'expédition.

```sql
SELECT COUNT(*) AS lignes,
       COUNT_IF(STRPTIME(OrderDateKey::VARCHAR, '%Y%m%d')::DATE = OrderDate::DATE) AS cles_conformes
FROM dbo.FactInternetSales;
```


© 2026 iAlexMG. Tous droits réservés.

Observez. 60 398 lignes sur 60 398 : la clé `OrderDateKey`, un entier comme `20130131`, est bien la date de commande écrite chiffre à chiffre. On la convertit en texte, puis en date.

```
SELECT DATEDIFF('day', OrderDate, ShipDate) AS jours_avant_expedition,
       DATEDIFF('day', OrderDate, DueDate)  AS jours_avant_echeance,
       COUNT(*)                             AS lignes
FROM dbo.FactInternetSales
GROUP BY ALL;
```

Observez. Une seule ligne : 7 jours, 12 jours, 60 398 lignes. Toutes les commandes partent une semaine après avoir été passées et arrivent à échéance douze jours après. Une régularité aussi parfaite trahit des données de démonstration : dans une vraie base, cette requête serait le début d'une analyse, pas sa fin.

5.2 — Tronquer au mois, nommer le jour

Théorie : `Theorie_02_SQL.md`, section 6.

Objectif : les commandes par mois au premier trimestre 2013, et la convention du jour de semaine.

```
SELECT DATE_TRUNC('MONTH', OrderDate)   AS mois,
       COUNT(DISTINCT SalesOrderNumber) AS commandes
FROM dbo.FactInternetSales
WHERE OrderDate >= '2013-01-01' AND OrderDate < '2013-04-01'
GROUP BY ALL
ORDER BY mois;
```

Observez. 632 commandes en janvier, 1 421 en février, 1 690 en mars. Le filtre par intervalle demi-ouvert et le regroupement par `DATE_TRUNC` vont ensemble.

```
SELECT FullDateAlternateKey,
       EnglishDayNameOfWeek,
       DayNumberOfWeek,
       DAYOFWEEK(FullDateAlternateKey)    AS dayofweek,
       ISODOW(FullDateAlternateKey)       AS dayofweek_iso
FROM dbo.DimDate
WHERE FullDateAlternateKey BETWEEN '2013-03-17' AND '2013-03-23'
ORDER BY FullDateAlternateKey;
```

Observez. Le dimanche 17 mars 2013 vaut 1 dans `DimDate`, 0 pour `DAYOFWEEK` et 7 pour `ISODOW`. Trois conventions pour un même jour. Notez au passage que `BETWEEN` convient ici : `FullDateAlternateKey` est une date pure, sans heure.

5.3 — L'âge à l'embauche

Théorie : `Theorie_02_SQL.md`, sections 3 et 6.

Objectif : calculer l'âge des employés au jour de leur embauche.

© 2026 iAlexMG. Tous droits réservés.

```sql
SELECT FirstName, LastName, BirthDate, HireDate,
       DATEDIFF('year', BirthDate, HireDate) AS age_datediff
FROM dbo.DimEmployee
WHERE FirstName = 'Roberto' AND LastName = 'Tamburello';
```

Observez. Roberto Tamburello, né le 12 juin 1974, a été embauché le 11 juin 2007, la veille de ses 33 ans. `DATEDIFF` lui en donne 33 : il a compté les 33 passages au 1er janvier entre les deux dates, pas les anniversaires. Combien d'employés sont touchés ?

```sql
SELECT COUNT(*) AS employes,
       COUNT_IF(BirthDate + TO_YEARS(DATEDIFF('year', BirthDate, HireDate)::INTEGER)
                > HireDate) AS ages_faux
FROM dbo.DimEmployee;
```

Observez. 111 employés sur 296. Le test compte ceux dont l'anniversaire de l'année d'embauche tombe après la date d'embauche : pour eux, `DATEDIFF` a un an de trop. L'âge juste retire cette année :

```sql
SELECT FirstName, LastName,
       DATEDIFF('year', BirthDate, HireDate)
         - IF(BirthDate + TO_YEARS(DATEDIFF('year', BirthDate, HireDate)::INTEGER)
              > HireDate, 1, 0) AS age_reel
FROM dbo.DimEmployee
WHERE FirstName = 'Roberto' AND LastName = 'Tamburello';
```

Observez. 32 ans. L'expression est longue ; dans un vrai projet, on l'écrit une fois, dans une vue ou un modèle, et on ne la réécrit plus.

5.4 — Un horodatage, deux fuseaux, deux jours

Théorie : `Theorie_02_SQL.md`, section 6.

Objectif : voir que le jour d'un horodatage dépend du fuseau où on le lit, puis lire un epoch.

```sql
SELECT CURRENT_SETTING('TimeZone') AS fuseau_de_la_session;
```

Observez. Le fuseau de votre poste, `America/Toronto` au Québec. C'est lui qui affiche les `TIMESTAMPTZ`, et c'est lui que prend un `CAST` en `DATE` qui ne dit pas le sien.

```sql
SELECT COUNT(*) AS lignes,
       COUNT_IF(CAST(OrderDate AT TIME ZONE 'UTC' AT TIME ZONE 'America/Toronto' AS DATE)
                <> CAST(OrderDate AS DATE)) AS changeraient_de_jour
FROM dbo.FactInternetSales;
```

Observez. 60 398 sur 60 398. `OrderDate` est une heure de mur, toujours minuit, sans fuseau. Le premier `AT TIME ZONE` dit « cette heure de mur est en UTC », le second « affichez-la à Montréal » : si la source avait écrit ses dates en UTC, chaque vente serait tombée la veille à Montréal, à 19 h ou à 20 h selon la saison. La première vente, le 29 décembre 2010, deviendrait le 28 à 19 h. Rien dans la colonne ne dit laquelle des deux lectures est la vraie : seule la source le sait.

© 2026 iAlexMG. Tous droits réservés.

```sql
SELECT TO_TIMESTAMP(1769895000) AS depuis_des_secondes,
       EPOCH_MS(1769895000000)  AS depuis_des_millisecondes;
```

Observez. Le même instant, compté en secondes puis en millisecondes, sort une fois en `2026-01-31 16:30:00-05`, un instant à l'heure de Montréal, une fois en `2026-01-31 21:30:00`, une heure de mur en UTC sans fuseau. Les deux sont justes, et rangées côte à côte dans une table elles sembleraient à cinq heures l'une de l'autre.

```sql
SELECT TIMESTAMPTZ '2026-11-01 05:30:00+00' AT TIME ZONE 'America/Toronto' AS premier,
       TIMESTAMPTZ '2026-11-01 06:30:00+00' AT TIME ZONE 'America/Toronto' AS second;
```

Observez. Deux instants à une heure d'écart, et la même heure de mur, `2026-11-01 01:30:00`, deux fois. C'est la nuit du retour à l'heure normale : une source qui écrit en heure locale sans fuseau range ces deux mesures sous le même horodatage.

Partie 6 — Agréger

6.1 — Ce que compte chaque `COUNT`

Théorie : `Theorie_02_SQL.md`, section 7.

Objectif : distinguer lignes, valeurs présentes et valeurs différentes.

```sql
SELECT COUNT(*)                 AS clients,
       COUNT(MiddleName)        AS avec_second_prenom,
       COUNT(DISTINCT LastName) AS noms_de_famille
FROM dbo.DimCustomer;
```

Observez. 18 484 clients, 10 654 seconds prénoms, 375 noms de famille différents.

```sql
SELECT COUNT(*)                          AS lignes,
       COUNT(DISTINCT SalesOrderNumber) AS commandes,
       COUNT(DISTINCT CustomerKey)      AS clients
FROM dbo.FactInternetSales;
```

Observez. 60 398 lignes, 27 659 commandes, 18 484 clients. Une commande compte en moyenne un peu plus de deux lignes : un rapport qui annoncerait « 60 398 commandes » se tromperait du simple au double.

6.2 — Les règles du `GROUP BY`

Théorie : `Theorie_02_SQL.md`, section 7.

Objectif : la vente moyenne selon la distance domicile-travail du client.

© 2026 iAlexMG. Tous droits réservés.

```sql
SELECT c.CommuteDistance,
       COUNT(*)                    AS lignes,
       SUM(f.SalesAmount)          AS ventes,
       ROUND(AVG(f.SalesAmount), 2) AS vente_moyenne
FROM dbo.FactInternetSales AS f
JOIN dbo.DimCustomer       AS c ON c.CustomerKey = f.CustomerKey
GROUP BY c.CommuteDistance
ORDER BY vente_moyenne DESC;
```

Observez. Les clients qui habitent à moins d'un mille de leur travail ont la vente moyenne la plus élevée, 526,01 $ sur 21 307 lignes, et ceux qui en habitent à un ou deux milles la plus basse, 446,57 $. Ajoutez maintenant une colonne non agrégée, sans la regrouper :

```sql
SELECT c.CommuteDistance,
       c.EnglishOccupation,
       COUNT(*) AS lignes
FROM dbo.FactInternetSales AS f
JOIN dbo.DimCustomer       AS c ON c.CustomerKey = f.CustomerKey
GROUP BY c.CommuteDistance;
```

Observez. DuckDB refuse, et son message dit la règle : la colonne « must appear in the GROUP BY clause or must be part of an aggregate function ». `EnglishOccupation` n'est ni regroupée ni agrégée, et le moteur ne sait pas laquelle des professions afficher pour chaque distance. Remplacez `GROUP BY c.CommuteDistance` par `GROUP BY ALL` : la requête passe, et le grain change, une ligne par distance et par profession. `GROUP BY ALL` ne devine rien, il regroupe sur tout ce qui n'est pas agrégé. Vérifiez toujours que ce grain est bien celui que vous vouliez.

Les exercices du cours et leur corrigé vivent dans `Exercices_corriges_02_SQL.md`. Faites-les après avoir parcouru les deux labos.

Et ensuite ?

Passez au labo jumeau, `Labo_02_OLTP_SQL.md` : les mêmes questions sur PostgreSQL, où la division entière, le `CHAR` et l'alias dans le `WHERE` se comportent autrement. Le Cours 03 quitte ensuite la lecture pour l'écriture : créer des tables, y insérer, les modifier, et découvrir ce que chaque moteur garantit vraiment.

© 2026 iAlexMG. Tous droits réservés.